Primjena SOLID principa u modelu sistema

1. SRP – Single Responsibility Principle

Svaka klasa u sistemu ima jasno definisanu i jedinstvenu odgovornost.

- Klasa **Korisnik** sadrži osnovne podatke o korisnicima sistema.
- Klasa **Pacijent** predstavlja korisnika koji koristi usluge klinike.
- Klasa **Doktor** sadrži podatke o medicinskom osoblju i njegovim specifičnostima.
- Klasa **Administrator** upravlja sistemom i korisnicima.
- Klasa **Termin** modelira zakazani pregled i njegove podatke.
- Klasa **UslugaKlinike** opisuje medicinske usluge.
- Klasa **Raspored** definiše radno vrijeme doktora.
- Klasa **QRKod** predstavlja identifikaciju termina putem QR koda.
- Klasa **SistemZaSkeniranjeQRKoda** simulira vanjski uređaj za očitavanje QR kodova.

Na ovaj način svaka klasa ima jednu odgovornost, čime je zadovoljen SRP princip.

2. OCP – Open/Closed Principle

Sistem je otvoren za proširenje, a zatvoren za izmjene postojećih klasa.

- Klase **Pacijent**, **Doktor** i **Administrator** nasljeđuju klasu **Korisnik**, čime se omogućava dodavanje novih tipova korisnika bez izmjene postojeće klase.
- Moguće je dodati nove klase (npr. *MedicinskaSestra*) bez izmjene postojećeg modela.
- Enumeracije poput **StatusTermina** i **StatusUredjaja** omogućavaju proširenje dodavanjem novih vrijednosti bez izmjene strukture klasa.

Na ovaj način sistem se može proširivati bez narušavanja postojećeg dizajna.

3. LSP – Liskov Substitution Principle

Izvedene klase mogu se koristiti umjesto bazne klase bez narušavanja funkcionalnosti sistema.

- Klase **Pacijent**, **Doktor** i **Administrator** su specijalizacije klase **Korisnik**.
- Svaki objekat ovih klasa može se posmatrati kao objekat tipa **Korisnik**.
- Zamjena bazne klase izvedenom klasom ne utiče negativno na rad sistema.

Time je osigurana pravilna primjena LSP principa.

4. ISP – Interface Segregation Principle

Sistem je podijeljen na više manjih, specijalizovanih cjelina umjesto jedne velike klase.

- Funkcionalnosti su raspoređene kroz različite klase (Termin, Raspored, QRKod, itd.).
- Klase ne sadrže nepotrebne operacije koje ne koriste.
- Na primjer, klasa **UslugaKlinike** sadrži samo podatke o uslugama, dok klasa **Termin** upravlja zakazivanjem i statusima termina.

Na ovaj način izbjegnuto je preopterećenje klasa i osigurana je njihova specijalizacija.

5. DIP – Dependency Inversion Principle

Sistem smanjuje zavisnost od konkretnih implementacija korištenjem apstrakcija.

- Klasa **SistemZaSkeniranjeQRKoda** predstavlja apstrakciju vanjskog uređaja za očitavanje QR kodova.
- Sistem ne zavisi direktno od konkretne implementacije hardvera, već od koncepta očitavanja QR koda.
- Klasa **QRKod** se koristi kao posrednik između sistema i vanjskog uređaja.

Na ovaj način postiže se fleksibilnost i mogućnost zamjene implementacije bez izmjene ostatka sistema.